Observability and local storage

Analytics, crash reporting, alerts, local storage, and app lifecycle

Dinu-Ștefan Rusu ADMD

What you should leave with

Production monitoring

Log the events, traces and crash reports that describe a device you will never touch.

Metrics and alerting

Crash-free users per version, p95 instead of the average, and alerts that have an owner.

Local storage

Preferences, secure storage, a SQL database, files, and a cache with a time to live.

Secure storage

What belongs in the keychain, what belongs on the server, and what to clear on sign-out.

1

Observability

Events, metrics, crashes, alerts

Contents

		Page
1	**Observability**	3
2	Local storage	15
3	App lifecycle	27

Mobile observability challenges

- The code runs on hardware you have never seen: thousands of Android models, several OS versions, networks that drop mid-request, devices throttled in a pocket.
- You cannot attach a debugger to a user's phone. There is no server log to grep, because the log is on the device and you have no access to it.
- Releases are staged and users update slowly, so several versions of your app are live at the same time and one of them is months old.

Monitoring tells you that something is wrong. Observability is having enough signal to work out why, without shipping a build to find out.

Types of observability signals

Events

What users did. Named actions read as a funnel, delayed by hours.

Metrics

How slow and how often. Traces and percentiles for the paths that matter.

Crashes

What broke, with a stack trace.

Alerts

Who finds out, and when.

Each layer answers a different question.

Logging an analytics event

```dart
final analytics =
    FirebaseAnalytics.instance;

// snake_case, at most 40 characters.
await analytics.logEvent(
  name: 'checkout_started',
  parameters: {
    'cart_value_ron': 249.90,
    'item_count': 3,
    'payment_method': 'card',
  },
);
```

Some events are free.

`first_open`, `session_start`, `screen_view` and `app_remove` are collected for you. Everything else is a deliberate decision.

Parameter values are strings or numbers. Wire `FirebaseAnalyticsObserver` into `MaterialApp` once and every route change becomes a screen view.

Event naming and personal data

- Name events for the funnel you want to read later. `checkout_started`, then `payment_submitted`, then `purchase_complete` is a chart. Three unrelated names are not.
- Never put personal data in a parameter: no emails, no names, no free text a user typed. That is a legal question about personal data.
- Events are aggregated and delayed by hours. They answer how many users, never what happened to this one user.

Analytics is not a log. When you need one user’s story you need a crash report with breadcrumbs, and that is the next slide.

Catching crashes in Flutter

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  final crash = FirebaseCrashlytics.instance;
  FlutterError.onError = crash.recordFlutterFatalError;
  PlatformDispatcher.instance.onError = (error, stack) {
    crash.recordError(error, stack, fatal: true);
    return true;
  };
  runApp(const App());
}
```

The first handler catches framework errors. The second catches async errors outside it. Reports upload on the next launch.

Reporting caught errors

```
try {
  await api.pay(cart);
} catch (e, s) {
  await crash.recordError(
      e, s, reason: 'checkout');
}
crash.log('card selected');
crash.setCustomKey('cart_items', 3);
crash.setUserIdentifier(hashedId);
```

A handled failure is still a report. Use it for the paths you catch on purpose: a payment the server refused, a response that failed to parse, a retry that gave up.

`log` adds breadcrumbs to the next report. Custom keys add state. Set a hashed identifier, never an email.

Report the errors you catch.

Crash-free users

99.9%
crash-free sessions, the number vendors like to quote

99.5%
crash-free users, a common floor for a consumer app

1 in 200
users hit a crash at that floor

- It counts people, not events: one user crashing fifty times still costs you one user. Sessions are always the higher number, because most sessions are short and uneventful.
- Read it per app version, never in aggregate. A bad release hides inside a healthy overall number for days.

Act on crash-free users for the version you are rolling out. If it drops, halt the rollout or flip the feature flag from Lecture 8.

Latency percentiles

Number	What it tells you
p50, the median	The typical request. The last number to move.
p95	One request in twenty. Where users start to notice.
p99	The tail: cold starts, retries, weak networks.
the average	One 30-second timeout skews it. No user experiences it.

Write the budget down in words: feed load p95 under 1.5 s. Below roughly a thousand samples, p99 is noise.

A percentile describes a defined share of your users.

Measuring latency in the app

```dart
final t = FirebasePerformance.instance
    .newTrace('feed_load');
await t.start();
await loadFeed();
t.putMetric('items', items.length);
await t.stop();
```

Some traces cost you nothing.
App start, app in foreground and app in background are captured as soon as the package is added. Per-screen rendering is not: Flutter draws into one native view.

Dart HTTP calls are not traced either. `dart:io` bypasses the instrumented native stack, so time the call yourself.

Give every trace a budget before you read it.

Alerting guidelines

Signal	Fire when	Why that threshold
Crash-free users	it drops on the new version	one user in 200 is an outage
New fatal issue	new in the build you shipped	regressions, not the old backlog
p95 latency	it doubles week over week	relative change survives growth
Funnel conversion	purchases drop, starts hold	a feature breaks without crashing
Everything else	never: use a dashboard	an alert nobody acts on is ignored

Every alert needs an owner and an action.

Incident walkthrough

Four minutes after a ten percent rollout of a new checkout.

14:02 Analytics

Checkout starts look normal, but purchases are down 40% on the new version.

14:05 Performance

p95 on the payment trace jumps from 0.4 to 5.2 seconds.

14:07 Crashlytics

A new fatal issue: null check operator used on a null value in PaymentResponse.

14:09 Remote Config

The flag goes back to false. Fix, ship, then ramp again from one percent.

Events show that something is wrong, metrics show where, crashes show why. The feature flag is what limits the damage without waiting for a store review.

2

Local storage

Preferences, secrets, a database, files, a cache

Contents

		Page
1	Observability	3
2	**Local storage**	15
3	App lifecycle	27

Storage options

Store	Good for	Not for
shared_preferences	key and value settings	secrets, queries, anything large
flutter_secure_storage	tokens and keys	bulk data of any kind
sqflite or Drift	rows you query and watch	binary blobs, one-off flags
Files via path_provider	images, exports, downloads	anything you search or filter
In-memory cache	objects for this session	anything that must survive exit

Size and shape pick the store. A list you filter belongs in a database. A single boolean does not.

Small settings with shared_preferences

```dart
final prefs = await SharedPreferences
    .getInstance();

await prefs.setBool('dark_mode', true);
await prefs.setString('tab', 'feed');
await prefs.setStringList('recent', ids);

final dark = prefs.getBool('dark_mode')
    ?? false;
await prefs.remove('tab');
```

A key and a value, nothing else. It wraps NSUserDefaults on iOS and SharedPreferences on Android. Values are bool, int, double, String or a list of strings.

Reads are synchronous once the file is loaded, so keep it small. Nothing here is encrypted. This API is legacy: new code should use SharedPreferencesAsync.

Settings, not application state. The user's preferences belong here. The data your screens are made of belongs in a database.

Secrets with flutter_secure_storage

```dart
const storage = FlutterSecureStorage();

await storage.write(
    key: 'refresh', value: token);

final token = await storage.read(
    key: 'refresh');

await storage.delete(key: 'refresh');
await storage.deleteAll();  // sign out
```

The platform holds the key. iOS and macOS use the Keychain. On Android the key lives in the Keystore and the values are stored as AES-GCM ciphertext.

Every call is asynchronous and crosses a platform channel. Read the token once at start-up, keep it in memory, write it only when it changes.

Clear it on sign-out. A token left behind means the next person holding the device is still signed in as your user.

Sensitive data

- Passwords. Do not store one at all: keep the token the server issued.
- Access tokens, refresh tokens and API keys. Anything that grants access to something.
- Personal data you do not need on the device: an address, a phone number, a document scan.
- Card details. The payment provider holds those, and your app never sees them.

⚠ Assume the file is readable

On a rooted or jailbroken device, and inside a device backup, the app sandbox is not a boundary.

Log lines count as storage. A token printed once ends up in a bug report attached by a user.

Put in encrypted storage only the few values you cannot avoid keeping on the device.

Structured data with sqflite

```
final db = await openDatabase(
  join(await getDatabasesPath(), 'todos.db'),
  version: 1,
  onCreate: (db, v) => db.execute(
    'CREATE TABLE todos(id TEXT PRIMARY KEY, title TEXT, done INTEGER)'),
);
await db.insert('todos', {'id': id, 'title': title, 'done': 0});
final rows = await db.query('todos',
    where: 'done = ?', whereArgs: [0], orderBy: 'title');
final todos = rows.map(Todo.fromMap).toList();
```

SQLite on both platforms, driven with SQL strings. There is no boolean column, so a flag is an integer. Always pass values through `whereArgs`.

Drift table definition

```
class Todos extends Table {
  TextColumn get id => text()();
  TextColumn get title => text().withLength(min: 1)();
  BoolColumn get done => boolean().withDefault(const Constant(false))();
  @override Set<Column> get primaryKey => {id};
}
@DriftDatabase(tables: [Todos])
class AppDb extends _$AppDb {
  @override int get schemaVersion => 1;
}
```

Drift sits on the same SQLite file and generates a typed query API from these declarations. The generator is `build_runner`, as for JSON in Lecture 7.

Watched queries in Drift

```
Stream<List<Todo>> watchOpen() =>
    (select(todos)
      ..where((t) =>
          t.done.equals(false)))
    .watch();

Future<void> toggle(Todo t) =>
    update(todos).replace(
        t.copyWith(done: !t.done));
```

A query that emits again. `watch()` returns a Stream that re-runs whenever a write touches the tables it reads. Hand it to a StreamBuilder or to the bloc from Lecture 6.

A schema change means a new version number and a migration step, tested against a database file from the previous release.

The database is the source of truth and the screen is a view of it. Write to the database, and let the stream update the widget for you.

Files with path_provider

```dart
final dir = await getApplicationDocumentsDirectory();
final file = File('${dir.path}/export.json');

await file.writeAsString(jsonEncode(data));
final text = await file.readAsString();
if (await file.exists()) await file.delete();

final tmp = await getTemporaryDirectory();
```

`dart:io` does not exist on the web, so guard the import if the app targets a browser.

Store the file name, not the absolute path. Ask path_provider again on every launch: the container directory can change when the app is updated.

App directories and cleanup

Call	Holds	Lifetime
getApplicationDocumentsDirectory	user files	survives updates, kept in backups
getApplicationSupportDirectory	app data	survives updates, not user visible
getTemporaryDirectory	caches	the system may delete it any time
getExternalStorageDirectory	shared files	Android only, outside the sandbox

Anything in the temporary directory can vanish between launches.

In-memory cache with TTL

```
class Cached<T> {
  Cached(this.value) : at = DateTime.now();
  final T value;
  final DateTime at;
  bool fresh(Duration ttl) => DateTime.now().difference(at) < ttl;
}
final _cache = <String, Cached<Feed>>{};
Feed? readFeed(String key, Duration ttl) {
  final e = _cache[key];
  return (e != null && e.fresh(ttl)) ? e.value : null;
}
```

A cache with no expiry is a memory leak: give every entry a time to live and a maximum size.

Repository read strategy

1. Return the cached value if it is still fresh. No await, no disk, no network, no spinner.
2. Otherwise read the database and return that immediately. The screen shows real content even with the network off.
3. Then fetch from the network, write the result to the database, and let the watched query push the update to the screen.

The bloc from Lecture 6 sees none of this. It asks the repository for todos and receives a stream.

One class owns the storage decision. If a widget opens a database or reads a preference, the layering has already failed.

3 App lifecycle

Contents

		Page
1	Observability	3
2	Local storage	15
3	**App lifecycle**	27

Lifecycle states

resumed

Visible and taking input. The only state in which animations and camera work belong.

inactive

Visible but not focused: an incoming call, the app switcher, a system dialog.

paused

Not visible. The process may be killed after this.

detached

The engine is alive with no view attached.

The paused state is your last reliable save point. Anything the user would be annoyed to lose must be on disk by then.

Saving when the app is paused

```
class _HomeState extends State<Home> with WidgetsBindingObserver {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
  }
  @override
  void didChangeAppLifecycleState(AppLifecycleState s) {
    if (s == AppLifecycleState.paused) _saveDraft();
  }
}
```

Remove the observer in `dispose` too, or the State object outlives its widget.

Storage checklist

- [ ] No password, token or key in preferences, in a file, or in a log line.
- [ ] Sign-out clears secure storage, the database and every in-memory cache.
- [ ] Every schema change has a migration, tested against a database from the previous release.
- [ ] Every cache entry has a time to live, and the cache has a maximum size.
- [ ] Draft state is written when the app is paused, not only when the user leaves the screen.
- [ ] The app opens and shows real content with the network turned off.

Offline-first sync and conflict resolution are MEC Lectures 8 and 9. This lecture stops at one device.

Summary

1. **The app has to report on itself.** Events, metrics, crashes and alerts, because you own none of the devices it runs on.
2. **Act on crash-free users for the version you are rolling out.** Read p95 for speed, and never the average.
3. **Size and shape pick the store.** Preferences for settings, secure storage for secrets, a database for anything you query.

Tasks for this week

This week

- Wire both crash handlers into the lab app, force one crash, and find the report.
- Log one analytics event that matters to your app, with no personal data in it.
- Move the todo list into a database and drive the screen from a watched query.
- Turn the network off, open the app, and write down every screen that shows nothing.

Further reading

Crashlytics for Flutter
docs.flutter.dev, persistence cookbook
drift.simonbinder.eu
pub.dev, flutter_secure_storage

The Drift site carries the migration guide. Read it before you change a table for the second time.

The lab app carries the third item forward. Show the schema in the lab even if the database half is unfinished, so it is checked early.

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel